Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

EcoRouting Engine: Hazard-Aware Path Optimization and Fill-Rate-Adaptive Waste Scheduling

Student/Team Information

Team Name: *Team #* *PBL Mentor:*	*EcoGraph* *DSCPP-III-2026-T186* *Mr. Navneet Rajput*
Team member 1 (Team Lead)	*Jain, Tejas* *Student ID: 2510011541* *Roll No. 2028228, Sec A* *aispace101@gmail.com* 
Team member 2	*Negi, Apoorv* *Student ID: 2510011088* *Roll No. 2027640, Sec D* *apoorvnegi17@gmail.com* 
Team member 3	*Nigam, Yash* *Student ID: 2510010441* *Roll No. 2028289, Sec H* *nigamyash07@gmail.com* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

Anyone who has watched a municipal garbage truck on its route notices the same thing: it drives the same streets every day, in the same order, whether or not there is anything to collect. Routes are fixed by habit rather than the actual shortest path between the depot, the homes on the list, and the disposal sites, wasting fuel, driver time, and public money. Two problems compound this. Hazardous waste – medical sharps, expired chemicals, batteries – is often collected the same way as ordinary household waste, even though it should be picked up immediately and kept off residential and commercial streets on its way to disposal; delays or careless routing here are a genuine safety and environmental risk. Second, every disposal site is treated as if it fills at the same rate, so a fixed weekly schedule either visits a disposal site that is nowhere near full or leaves an overflowing one unattended. We wanted a system that treats the city as the graph it actually is, and routes, restricts, and schedules collection accordingly.

Our project is a working C++ backend that models the city as a weighted, zone-tagged graph and answers three questions on demand: the shortest route to collect from a given set of homes, which locations are actually due today based on fill-rate, and how to route hazardous waste safely away from residential streets – all exposed through a REST API and shown live on a PixiJS frontend that animates the computed route. The system is hosted at `ecograph-xl8q.onrender.com`.

Updated Project Approach and Architecture

Design. One C++17 process serves the REST API and the front end on one port. `controllers.cpp` handles HTTP and JSON behind a mutex; domain classes (`Graph`, Dijkstra, `Scheduler`, classifier, route builders, `Simulation`) work on an in-memory adjacency-list city loaded from `city.json`.

Routing. `RouteStrategy` is an abstract base class; its Priority subclass sets the stop order (priority mode: the fullest home comes first).

Protocol. REST over HTTP with JSON, 15 endpoints, one error format.

Front end. Plain JavaScript with no build step, plus a day-by-day simulation and a limited fleet.

Libraries. cpp-httplib, nlohmann/json, PixiJS.

Deployment. A multi-stage Docker image compiles the server, then copies only the binary, `city.json` and the front end into a slim runtime image. It runs as one long-lived container on Render, with the front end on the same origin as the API.

System overview

(diagrams (a)–(g) on the next pages)

1. The clock moves forward one day.

2. A home is due once the days since its last collection reach capacity / fill rate (1–14 days); the fullest home comes first (c).

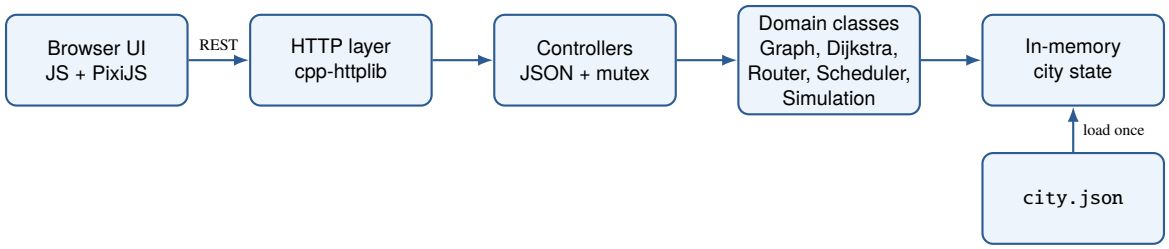
3. Due disposal sites are emptied, hazardous homes go to the hazard trucks (f), and normal homes are ordered and packed onto trucks before routing (e).

4. Each truck chains shortest paths in its list order:

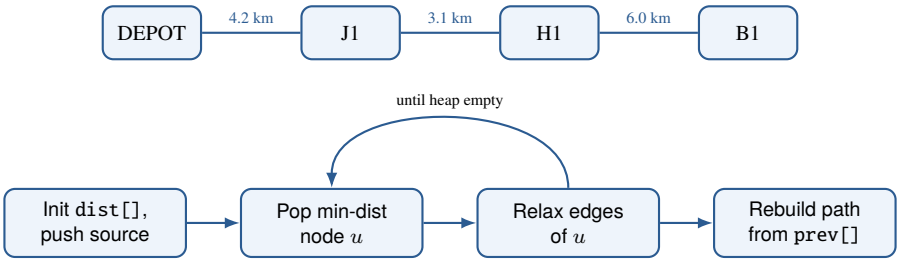
```
current = DEPOT;  visited = { }
for each home in order:
  skip it if in visited
  path = Dijkstra(current → home);  if none, mark unreachable
  empty later stops on the path, add them to visited          (en-route pickup, (d))
  empty home, add to visited;  current = home
then visit the nearest disposal site for each waste type collected
```

5. Served homes get “last collected = today”; unreachable ones are reported as failed, and carried-over homes rank higher tomorrow.

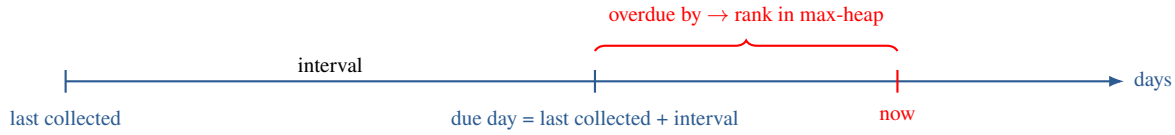
(a) System Pipeline



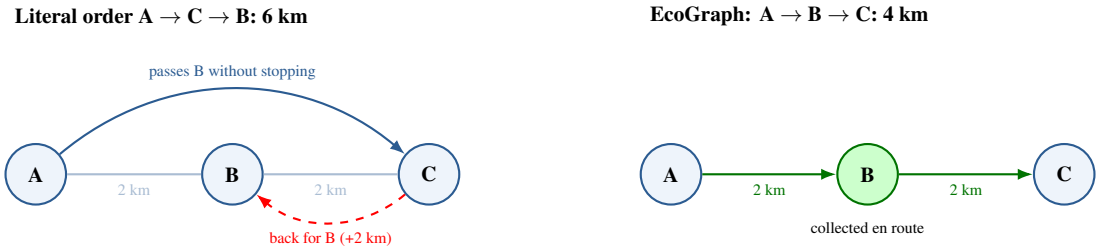
(b) Graph Storage and Dijkstra



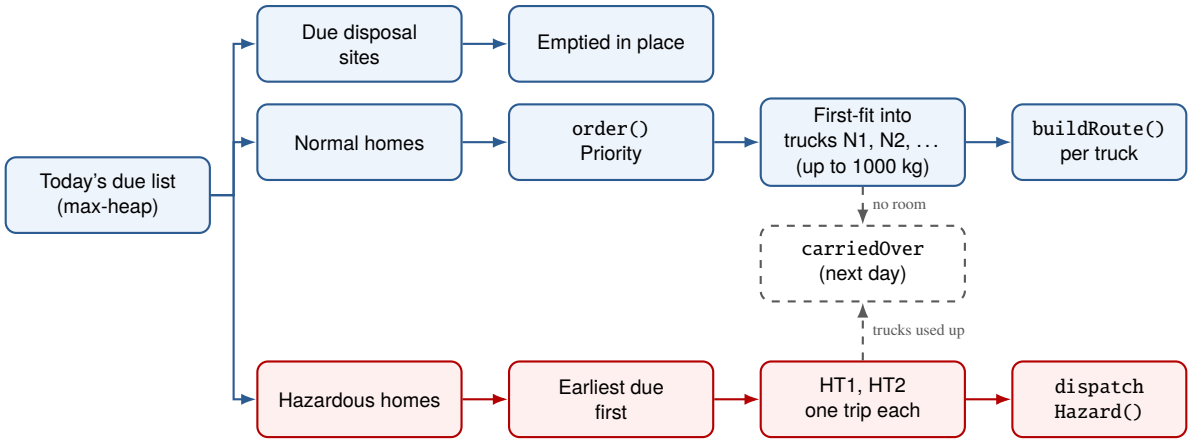
(c) Fill-Rate Scheduling



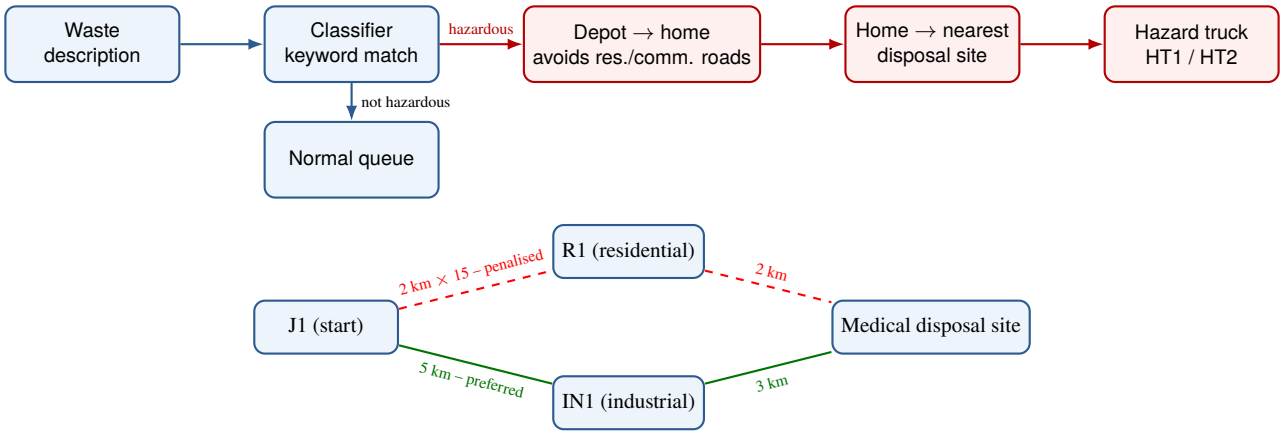
(d) En-Route Pickup



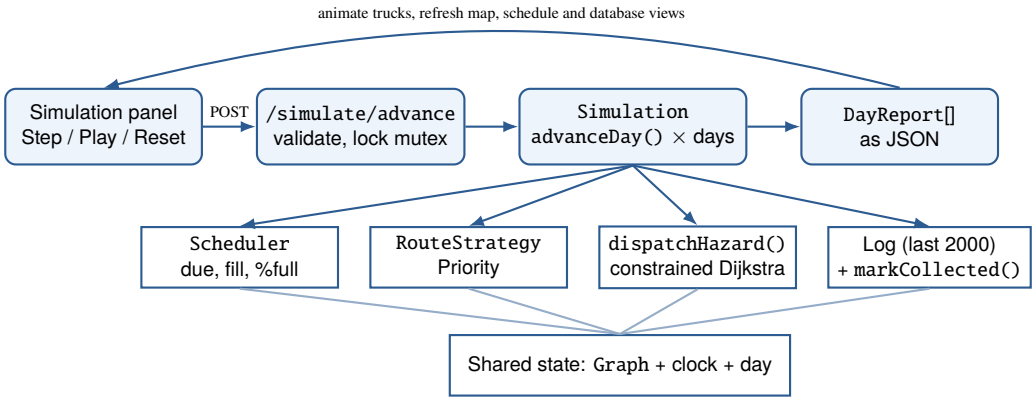
(e) Truck Fleet and Capacity



(f) Automatic Hazard Dispatch and Risk-Weighted Dijkstra



(g) Simulation Architecture



Tasks Completed

Task Completed	Team Member
Weighted, zone-tagged city graph as an adjacency list, with a JSON loader for the Dehradun map (70 nodes, 107 roads)	Tejas Jain (Lead)
HTTP server on cpp-httplib serving the front end and 15 REST endpoints, with one error format and a mutex on shared state	Tejas Jain
Plain and zone-constrained Dijkstra with path rebuilding	Tejas Jain
Priority route builder (fullest home first) with en-route pickup and unloading at the nearest disposal site	Tejas Jain
Keyword waste classifier and automatic hazard dispatch to the nearest medical or chemical disposal site, keeping off residential and commercial roads wherever another road exists	Tejas Jain
Fill-rate scheduler with a max-heap of due locations	Tejas Jain
Day-by-day simulation with limited fleet, truck capacity, carry-over and a collection log	Tejas Jain, Apoorv Negi, Yash Nigam
Unit test suites for the backend, plus a check of the city data	Yash Nigam, Apoorv Negi, Tejas Jain
README with build steps, folder layout and API reference	Apoorv Negi, Yash Nigam
PixiJS map view with truck route animation, hazard-route styling and a shape legend	Yash Nigam, Tejas Jain, Apoorv Negi
Schedule, simulation and database views and the light/dark theme	Tejas Jain, Yash Nigam
Multi-stage Dockerfile and `.dockerignore`	Yash Nigam, Tejas Jain
Deployment on Render as one long-lived container, health check on `/health`, automatic redeploy from `main`	Tejas Jain
`PORT`, `CITY_FILE` and `FRONTEND_DIR` set in the image; front end on the API's origin with `ECO_API_BASE` empty	Tejas Jain, Yash Nigam
One-line `docker run` instructions in the README	Yash Nigam
Live demo link in the README	Tejas Jain
Front-end improvements	Tejas Jain
Code cleanup	Tejas Jain, Apoorv Negi
Phase-I and Phase-II reports and presentations	Apoorv Negi, Yash Nigam

Challenges/Roadblocks

Road classes beyond residential and commercial. If roads were tagged only as residential or commercial, the hazard truck would have no safe road to prefer. Real cities also have main roads between localities and industrial belts where disposal sites sit, so we added *arterial* and *industrial* road classes. The hazard truck's route weights residential roads 15× and commercial roads 10×, so it crosses the city on these classes and enters a neighbourhood or a market only for the stretch it cannot avoid.

Trucks driving past houses on their own list. At first the router followed the scheduler's order literally: given A, C, B with B on the road from A to C, the truck drove past B, reached C, then turned back for B. We added en-route pickup: each leg's path is checked for later stops, which are emptied on the way and skipped when the list reaches them.

Scheduling that never showed its effect. With unlimited trucks, every due home was served every day, so nothing ever overflowed, and the priority order only changed the stop order, never who got served. We added a limited fleet with 1000 kg per truck and carry-over to the next day. Since an overflowing home heavier than a whole truck would never fit, an empty truck always accepts its first home.

Shared state across requests. cpp-httplib handles requests on several threads, and routing, collection and the simulation all change the same graph, so all shared state sits behind one mutex.

Compiler on Windows. The old MinGW g++ could not build cpp-httplib, which needs proper thread support, so we moved to GCC 13.

Project Outcome/Deliverables

We are handing over:

- A C++17 server that loads a city graph and answers routing, hazard dispatch, classification, scheduling and simulation requests over a REST/JSON API.
- A browser front end that draws the Dehradun map, animates each truck's route, highlights hazardous trips, and lets you run the city forward one day at a time.
- Unit tests for every core module, plus a sample city file someone else can modify.
- A hosted demo at `ecograph-xl8q.onrender.com`, the README and API docs, and the Phase-II and Phase-III reports and presentations.

The real outcome we care about is simple: given a list of homes, the system should produce a shorter route than a fixed one would, keep hazardous waste off residential and commercial streets wherever another road exists, and stop sending trucks to disposal sites that are nowhere near full.

Progress Overview

The project is now 100% complete. At Phase-II it stood at roughly 80%, with the deployment, further front-end improvements and some code cleanup left to do; all three are now finished: the graph model, both versions of Dijkstra, the route builders, the waste classifier, the fill-rate scheduler, the simulation, the REST API and the front end all work, and the same system runs online at the hosted link.

Nothing in the build is behind schedule. Phase-I planned for deployment in Phase-III, and that is when it was done. The day-by-day simulation and limited truck fleet went beyond the original plan, because a single route on a static map didn't show whether the scheduling actually worked.

The live link is in the README, and the system is ready for the Phase-III presentation and final demo.

Codebase Information

Repository: `github.com/Space-Fighter/EcoGraph` **Branch:** `main`

Hosted demo: `https://ecograph-xl8q.onrender.com`

Layout: `server/` (C++ source, headers, tests, `data/city.json`) and `frontend/` (HTML, CSS, JS modules).

Key files

File	What it does
`server/src/main.cpp`	Starts the server; reads the port, city file and front-end folder
`server/src/controllers.cpp`	The 15 REST endpoints: parses JSON, calls the domain classes, guards shared state with a mutex
`server/src/graph.cpp`	City graph: nodes and zone-tagged roads stored as an adjacency list
`server/src/dijkstra.cpp`	Plain and zone-constrained shortest path
`server/src/scheduler.cpp`	Collection interval, due, overdue and % full; max-heap of due locations
`server/src/waste_classifier.cpp`	Keyword rules: medical, chemical, recyclable, compost (else general)
`server/src/router.cpp`	Priority route builder, en-route pickup, nearest disposal site unloading, hazard dispatch
`server/src/simulation.cpp`	Advances days, loads trucks, runs routes and hazard trips, keeps the collection log
`server/src/city_loader.cpp`	Reads `city.json` into the graph
`server/data/city.json`	The Dehradun city: localities, nodes, roads, zones, fill rates
`server/tests/`	Unit test programs (Dijkstra, domain, simulation, generator, city data)
`frontend/js/graph-view.js`	PixiJS map and truck route animation
`frontend/js/schedule-view.js`, `simulation-view.js`, `database-view.js`	Schedule, simulation and database panels
`frontend/js/api.js`	Thin `fetch` wrapper around the REST API
`Dockerfile`, `.dockerignore`, `render.yaml`	Container image and Render deployment settings

Important commits and the modules they added

Commit	Modules / checkpoint
Graph model and Dijkstra	`graph`, `dijkstra` (plain and constrained), Dijkstra tests
REST API server, city data and front end	`scheduler`, `waste_classifier`, `router` (Priority), `controllers`, map view
Day-by-day simulation and database view	`simulation`, database view, simulation tests
Hazard truck dispatch and limited fleet	automatic hazard trucks, truck count and capacity in `simulation` and `router`
City mapped to Dehradun (km/kg units)	real coordinates and localities
Full README with API reference	README, API docs
Schematic locality layout	cleaner map layout of the localities
En-route pickup	en-route collection in `router`
Add Dockerfile (`5ab2551`)	Docker image and `docker run` instructions
Add Render deployment config (`0b8130f`)	`render.yaml`, updates to `Dockerfile` and `.dockerignore`

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Unit: Dijkstra (`test_dijkstra`)	Pass	Hand-solved graphs; unreachable target handled
Unit: domain (`test_domain`)	Pass	Classifier rules, scheduler intervals clamped to 1–14 days, Priority ordering and tie-breaks, constrained route avoids a forbidden road, hazard split
Unit: simulation (`test_simulation`)	Pass	Truck capacity limits, carry-over to next day, hazard trucks
Unit: generator (`test_generator`)	Pass	Generated nodes are unique, connected, reachable and placed inside their locality
Data: city file (`test_city_data`)	Pass	Checks the Dehradun city file, including its depot, localities and hazard homes
API checks on the hosted site (curl)	Pass	`/health`, `/graph`, `/schedule`, `/classify`, `/route`, `/simulate/advance` return the expected JSON; bad JSON gets a clean 400 error
Browser check on the hosted site	Pass	Map, truck animation and database view load and run; screenshots on the next pages

Deliverables Progress

- C++ backend: routing, hazard dispatch, classifier, scheduler, simulation, REST API – **Completed**
- Browser front end with map, route animation, schedule and database views – **Completed**
- Dehradun city data – **Completed**
- Unit tests (4 suites + city data check) – **Completed**
- README and API documentation – **Completed**, including the live link
- Hosted demo (Dockerfile + deployment on Render) – **Completed**
- Phase-II report and presentation – **Completed**
- Phase-III report – **Completed**; Phase-III presentation and final demo – **Pending**

Application Screenshots

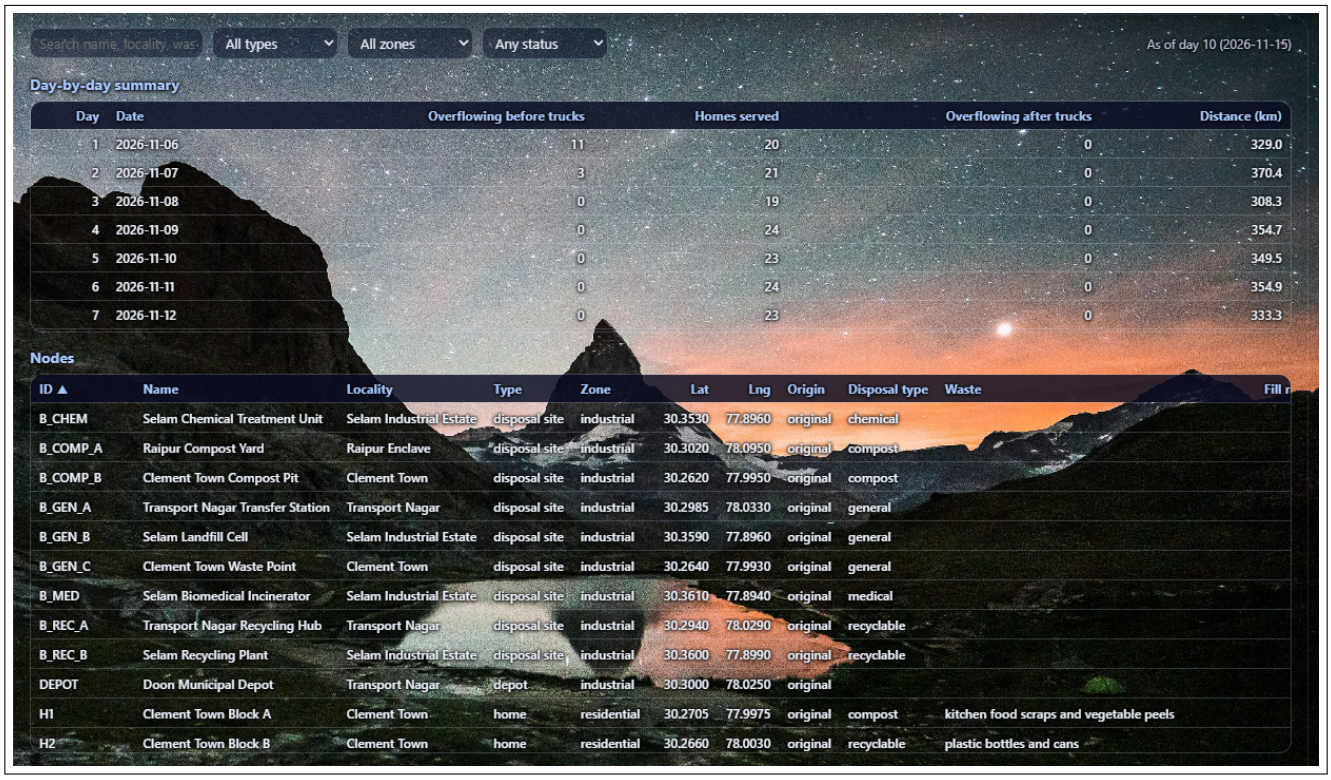
Taken from the hosted site, `ecograph-xl8q.onrender.com`.



Map view: the Dehradun city at the start of the simulation (Day 0).



Map view during Day 1: truck routes in blue, hazard routes in dashed red, and each truck's stops and distance in the simulation panel.


Search name, locality, was… All types All zones Any status As of day 10 (2026-11-15)

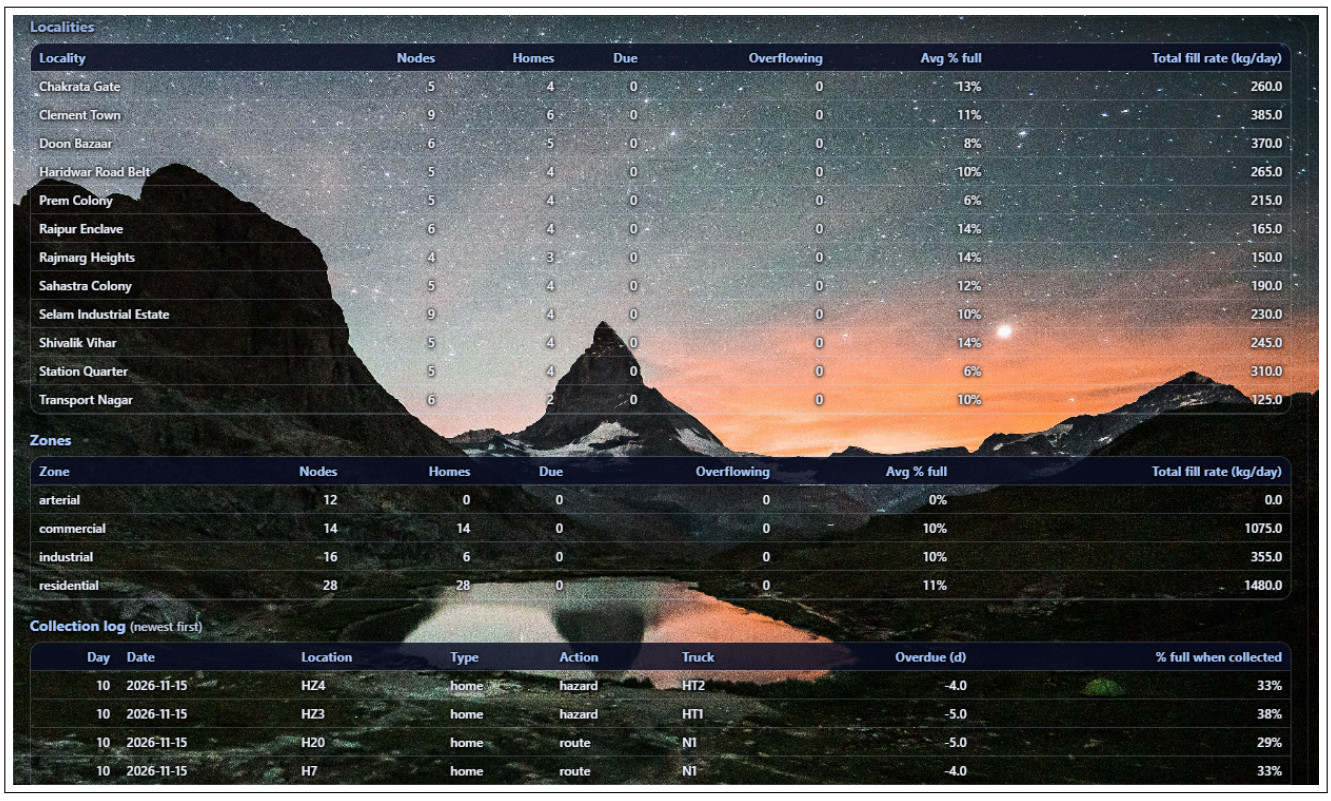
Day-by-day summary

Day	Date	Overflowing before trucks	Homes served	Overflowing after trucks	Distance (km)
1	2026-11-06	11	20	0	329.0
2	2026-11-07	3	21	0	370.4
3	2026-11-08	0	19	0	308.3
4	2026-11-09	0	24	0	354.7
5	2026-11-10	0	23	0	349.5
6	2026-11-11	0	24	0	354.9
7	2026-11-12	0	23	0	333.3

Nodes

ID ▲	Name	Locality	Type	Zone	Lat	Lng	Origin	Disposal type	Waste	Fill r
B_CHEM	Selam Chemical Treatment Unit	Selam Industrial Estate	disposal site	industrial	30.3530	77.8960	original	chemical		
B_COMP_A	Raipur Compost Yard	Raipur Enclave	disposal site	industrial	30.3020	78.0950	original	compost		
B_COMP_B	Clement Town Compost Pit	Clement Town	disposal site	industrial	30.2620	77.9950	original	compost		
B_GEN_A	Transport Nagar Transfer Station	Transport Nagar	disposal site	industrial	30.2985	78.0330	original	general		
B_GEN_B	Selam Landfill Cell	Selam Industrial Estate	disposal site	industrial	30.3590	77.8960	original	general		
B_GEN_C	Clement Town Waste Point	Clement Town	disposal site	industrial	30.2640	77.9930	original	general		
B_MED	Selam Biomedical Incinerator	Selam Industrial Estate	disposal site	industrial	30.3610	77.8940	original	medical		
B_REC_A	Transport Nagar Recycling Hub	Transport Nagar	disposal site	industrial	30.2940	78.0290	original	recyclable		
B_REC_B	Selam Recycling Plant	Selam Industrial Estate	disposal site	industrial	30.3600	77.8990	original	recyclable		
DEPOT	Doon Municipal Depot	Transport Nagar	depot	industrial	30.3000	78.0250	original			
H1	Clement Town Block A	Clement Town	home	residential	30.2705	77.9975	original	compost	kitchen food scraps and vegetable peels	
H2	Clement Town Block B	Clement Town	home	residential	30.2660	78.0030	original	recyclable	plastic bottles and cans	

Database view: day-by-day summary and the node table.


Localities

Locality	Nodes	Homes	Due	Overflowing	Avg % full	Total fill rate (kg/day)
Chakrata Gate	5	4	0	0	13%	260.0
Clement Town	9	6	0	0	11%	385.0
Doon Bazaar	6	5	0	0	8%	370.0
Haridwar Road Belt	5	4	0	0	10%	265.0
Prem Colony	5	4	0	0	6%	215.0
Raipur Enclave	6	4	0	0	14%	165.0
Rajmarg Heights	4	3	0	0	14%	150.0
Sahastra Colony	5	4	0	0	12%	190.0
Selam Industrial Estate	9	4	0	0	10%	230.0
Shivalik Vihar	5	4	0	0	14%	245.0
Station Quarter	5	4	0	0	6%	310.0
Transport Nagar	6	2	0	0	10%	125.0

Zones

Zone	Nodes	Homes	Due	Overflowing	Avg % full	Total fill rate (kg/day)
arterial	12	0	0	0	0%	0.0
commercial	14	14	0	0	10%	1075.0
industrial	16	6	0	0	10%	355.0
residential	28	28	0	0	11%	1480.0

Collection log (newest first)

Day	Date	Location	Type	Action	Truck	Overdue (d)	% full when collected
10	2026-11-15	HZ4	home	hazard	HT2	-4.0	33%
10	2026-11-15	HZ3	home	hazard	HT1	-5.0	38%
10	2026-11-15	H20	home	route	N1	-5.0	29%
10	2026-11-15	H7	home	route	N1	-4.0	33%

Database view: locality and zone summaries and the newest entries of the collection log.